

# Tutorial 16

## Generative Adversarial Network (GANs)

### Base GAN

#### Dataset

- We used the MNIST dataset which contains 60,000 grayscale images of handwritten digits from 0 to 9, each of size 28 by 28 pixels.
- The pixel values were normalized to fall between negative one and one so the generator output using Tanh would match the same range.
- Images were loaded in batches of 128 with shuffling enabled to ensure varied training.

#### Model Architecture

##### Generator

- The generator takes a random noise vector of 100 values and gradually transforms it into a 28 by 28 image through a series of fully connected layers.
  - Linear layer from 100 to 256 followed by LeakyReLU
  - Linear layer from 256 to 512 followed by LeakyReLU
  - Linear layer from 512 to 784 followed by Tanh, then reshaped to a 28 by 28 image

##### Discriminator

- The discriminator receives an image, flattens it to 784 values, and decides whether it is real or fake.
  - Linear layer from 784 to 512 followed by LeakyReLU and Dropout
  - Linear layer from 512 to 256 followed by LeakyReLU and Dropout
  - Linear layer from 256 to 1 followed by Sigmoid to output a probability

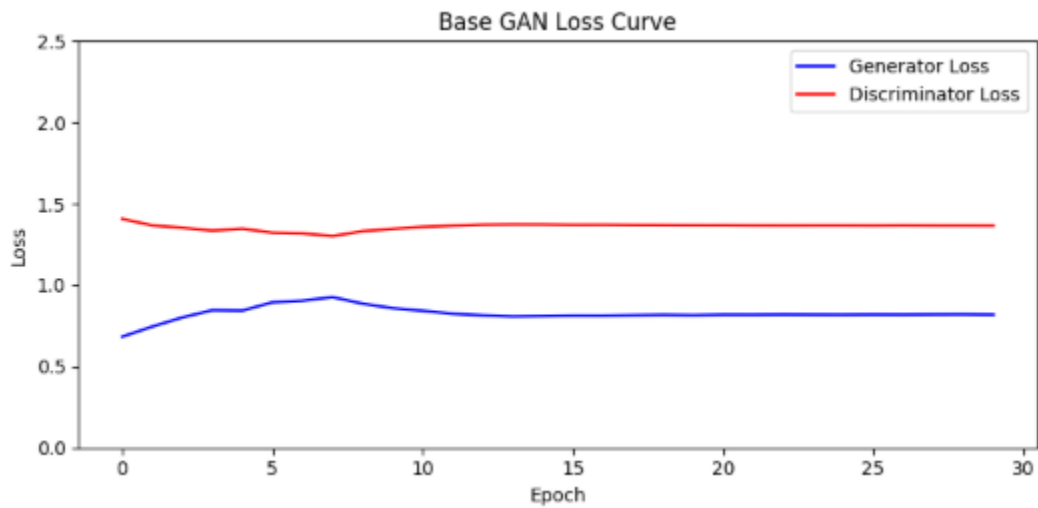
#### Training Setup

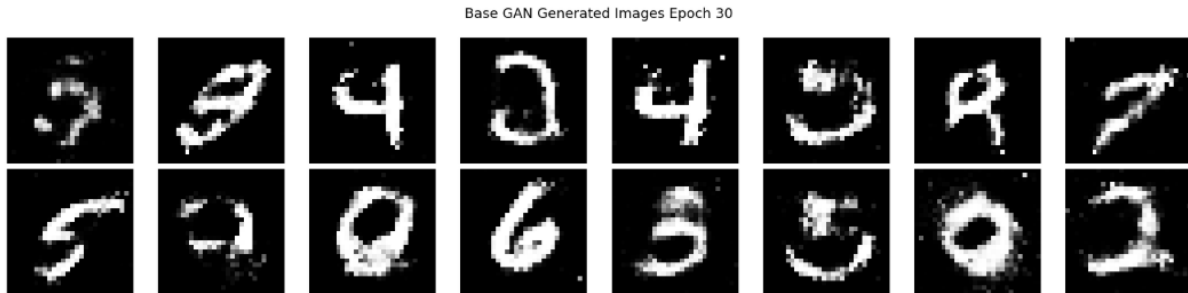
- Binary Cross Entropy loss was used for both the generator and discriminator.
- The generator was optimized with Adam at a learning rate of 0.0003 while the discriminator used a slower rate of 0.0001 to keep the two networks balanced.
- Real image labels were set to 0.9 instead of 1.0 as a smoothing technique to prevent the discriminator from becoming overconfident.
- The generator was updated twice for every one update of the discriminator so it had a fair chance to keep up.
- Training ran for 30 epochs with a batch size of 128.

## Results

```
***
=====
Training: Base GAN FC 30 Epochs
=====
Epoch [ 1/30] | D Loss: 1.4074 | G Loss: 0.6828
Epoch [ 2/30] | D Loss: 1.3674 | G Loss: 0.7452
Epoch [ 3/30] | D Loss: 1.3527 | G Loss: 0.8014
Epoch [ 4/30] | D Loss: 1.3353 | G Loss: 0.8459
Epoch [ 5/30] | D Loss: 1.3471 | G Loss: 0.8435
Epoch [ 6/30] | D Loss: 1.3223 | G Loss: 0.8945
Epoch [ 7/30] | D Loss: 1.3183 | G Loss: 0.9041
Epoch [ 8/30] | D Loss: 1.3014 | G Loss: 0.9265
Epoch [ 9/30] | D Loss: 1.3318 | G Loss: 0.8861
Epoch [10/30] | D Loss: 1.3460 | G Loss: 0.8584
Epoch [11/30] | D Loss: 1.3584 | G Loss: 0.8422
Epoch [12/30] | D Loss: 1.3661 | G Loss: 0.8241
Epoch [13/30] | D Loss: 1.3717 | G Loss: 0.8139
Epoch [14/30] | D Loss: 1.3733 | G Loss: 0.8078
Epoch [15/30] | D Loss: 1.3729 | G Loss: 0.8099
Epoch [16/30] | D Loss: 1.3711 | G Loss: 0.8120
Epoch [17/30] | D Loss: 1.3713 | G Loss: 0.8119
Epoch [18/30] | D Loss: 1.3701 | G Loss: 0.8146
Epoch [19/30] | D Loss: 1.3695 | G Loss: 0.8170
Epoch [20/30] | D Loss: 1.3690 | G Loss: 0.8149
Epoch [21/30] | D Loss: 1.3687 | G Loss: 0.8180
Epoch [22/30] | D Loss: 1.3676 | G Loss: 0.8178
Epoch [23/30] | D Loss: 1.3671 | G Loss: 0.8191
Epoch [24/30] | D Loss: 1.3674 | G Loss: 0.8183
Epoch [25/30] | D Loss: 1.3674 | G Loss: 0.8179
Epoch [26/30] | D Loss: 1.3668 | G Loss: 0.8190
Epoch [27/30] | D Loss: 1.3676 | G Loss: 0.8184
Epoch [28/30] | D Loss: 1.3668 | G Loss: 0.8197
Epoch [29/30] | D Loss: 1.3667 | G Loss: 0.8204
Epoch [30/30] | D Loss: 1.3662 | G Loss: 0.8184
```

Base GAN Loss





## Discussion

The training went smoothly from start to finish. The generator loss stayed in a healthy range between roughly 0.68 and 0.93 throughout the entire run, which is close to the theoretical ideal value of around 0.69. The discriminator loss settled at around 1.37 and barely moved, which tells us neither network was overpowering the other. Both curves flattened out by around epoch 12 and stayed consistent, which is exactly the kind of stable equilibrium we want to see in GAN training.

Looking at the generated images, the model learned to produce recognizable digit shapes by epoch 30. You can clearly make out numbers like 2, 3, 4, 7, 8, and 9 with different writing styles. Some images are a bit noisy or have slightly blurry edges, which is a known limitation of using only fully connected layers. These layers treat the image as a flat list of pixel values rather than understanding the spatial relationships between neighboring pixels, so finer details are harder to capture.

## Task 1: Deeper Architecture with More Epochs

### Changed Parameters

- The generator was made deeper by adding an extra layer and expanding the number of neurons at each stage, going from 100 inputs up through 256, 512, and 1024 before reaching the output.
- Batch normalization was added after each hidden layer in the generator to help the network learn more steadily and avoid the gradients becoming too large or too small.
- The discriminator kept the same structure but with dropout added for regularization.
- The discriminator learning rate was kept lower than the generator so it would not dominate the training.
- Training was run for 32 epochs, at which point the losses had fully stabilized and further training was not needed.

## Model Architecture

### Generator

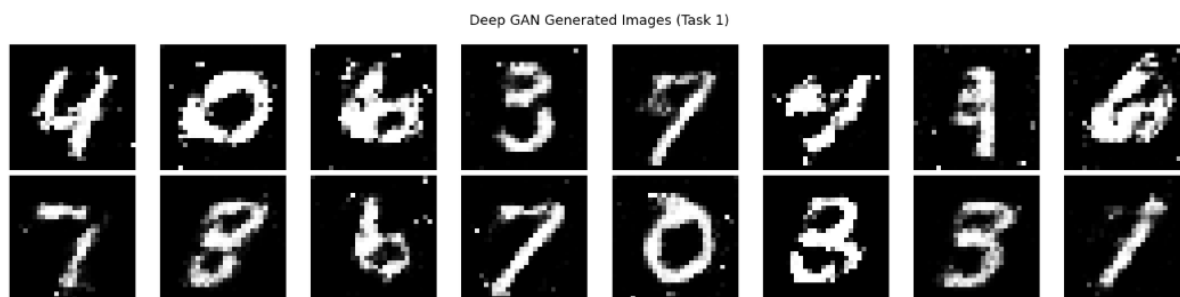
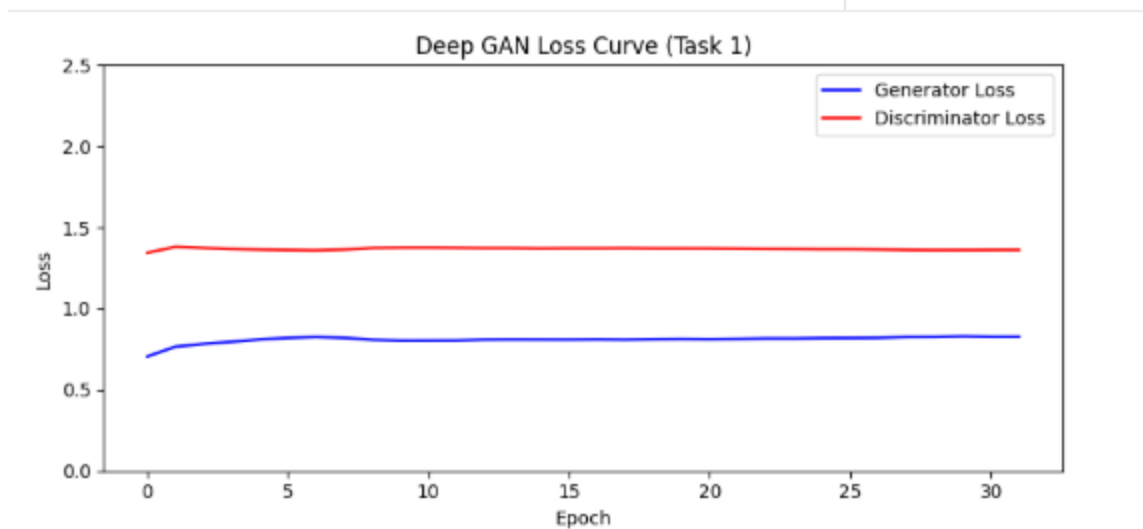
- A four layer fully connected network with batch normalization after each hidden layer.
  - Linear from 100 to 256, LeakyReLU, then batch normalization
  - Linear from 256 to 512, LeakyReLU, then batch normalization
  - Linear from 512 to 1024, LeakyReLU, then batch normalization
  - Linear from 1024 to 784, then Tanh to produce the output image

### Discriminator

- A three layer network with dropout applied after each of the first two layers.
  - Linear from 784 to 512, LeakyReLU, then Dropout
  - Linear from 512 to 256, LeakyReLU, then Dropout
  - Linear from 256 to 1, then Sigmoid

## Results

```
=====
Training: Deep GAN (More Layers, 50 Epochs)
=====
Epoch [ 1/50] | D Loss: 1.3441 | G Loss: 0.7052
Epoch [ 2/50] | D Loss: 1.3806 | G Loss: 0.7656
Epoch [ 3/50] | D Loss: 1.3736 | G Loss: 0.7826
Epoch [ 4/50] | D Loss: 1.3680 | G Loss: 0.7957
Epoch [ 5/50] | D Loss: 1.3636 | G Loss: 0.8100
Epoch [ 6/50] | D Loss: 1.3609 | G Loss: 0.8195
Epoch [ 7/50] | D Loss: 1.3590 | G Loss: 0.8259
Epoch [ 8/50] | D Loss: 1.3640 | G Loss: 0.8197
Epoch [ 9/50] | D Loss: 1.3734 | G Loss: 0.8080
Epoch [10/50] | D Loss: 1.3750 | G Loss: 0.8035
Epoch [11/50] | D Loss: 1.3753 | G Loss: 0.8037
Epoch [12/50] | D Loss: 1.3742 | G Loss: 0.8042
Epoch [13/50] | D Loss: 1.3729 | G Loss: 0.8088
Epoch [14/50] | D Loss: 1.3728 | G Loss: 0.8095
Epoch [15/50] | D Loss: 1.3712 | G Loss: 0.8092
Epoch [16/50] | D Loss: 1.3723 | G Loss: 0.8088
Epoch [17/50] | D Loss: 1.3721 | G Loss: 0.8099
Epoch [18/50] | D Loss: 1.3728 | G Loss: 0.8086
Epoch [19/50] | D Loss: 1.3714 | G Loss: 0.8107
Epoch [20/50] | D Loss: 1.3714 | G Loss: 0.8121
Epoch [21/50] | D Loss: 1.3711 | G Loss: 0.8107
Epoch [22/50] | D Loss: 1.3699 | G Loss: 0.8127
Epoch [23/50] | D Loss: 1.3687 | G Loss: 0.8158
Epoch [24/50] | D Loss: 1.3680 | G Loss: 0.8157
Epoch [25/50] | D Loss: 1.3664 | G Loss: 0.8183
Epoch [26/50] | D Loss: 1.3666 | G Loss: 0.8189
Epoch [27/50] | D Loss: 1.3644 | G Loss: 0.8202
Epoch [28/50] | D Loss: 1.3618 | G Loss: 0.8260
Epoch [29/50] | D Loss: 1.3604 | G Loss: 0.8268
Epoch [30/50] | D Loss: 1.3607 | G Loss: 0.8296
Epoch [31/50] | D Loss: 1.3615 | G Loss: 0.8273
```



## Discussion

The deeper architecture trained even more smoothly than the base model. The generator loss dropped quickly in the first few epochs and then leveled off at around 0.80, staying there consistently for the rest of training. The discriminator loss climbed slightly from its starting point and held steady at around 1.35, which means the two networks reached a good balance early and never drifted from it.

The images generated by this model are noticeably better than the base GAN. The digits look cleaner and sharper with much less noise in the background. Numbers like 0, 3, 4, 6, 7, and 8 are clearly legible and look like something a person could have written. The improvement comes mainly from the batch normalization layers, which keep the activations in a useful range throughout the network and allow the gradients to flow properly even through the extra layers. Training stopped at epoch 32 because the losses had already fully settled and there was no benefit to continuing further.

## Task 2: Convolutional GAN (DCGAN)

### Why Convolutional Layers

- The fully connected models in the previous tasks had no awareness of where pixels are located relative to each other. Every pixel was treated as an independent input with no connection to its neighbors.
- Convolutional layers fix this by scanning the image with small filters that learn local patterns like edges, curves, and corners. This makes the generator capable of producing much sharper and more structured images.
- DCGAN uses transpose convolutions in the generator to build up an image from a small starting point, and regular convolutions in the discriminator to analyze the image spatially.

### Stability Measures Used

- All convolutional and batch normalization weights were initialized from a normal distribution centered at zero with a small standard deviation, which is the standard starting point for this type of network.
- Spectral normalization was applied to every convolutional layer in the discriminator. This puts a hard limit on how strongly the discriminator can respond to any input, which stops it from becoming too powerful and causing the generator to collapse.
- Real image labels were set to 0.9 to prevent the discriminator from learning an overly sharp decision boundary.
- The discriminator used a learning rate of 0.0001 while the generator used 0.0002, so the generator always had a slight advantage in keeping up.
- The generator was updated twice for every discriminator update throughout training.
- Images were resized to 32 by 32 pixels so the convolutional strides would divide evenly at each layer.

### Model Architecture

#### Generator

- The generator starts from a noise vector and uses transpose convolutions to progressively double the spatial size at each step until it reaches the final 32 by 32 output.
  - Transpose convolution from 100 channels to 256, batch normalization, ReLU, output size 4 by 4
  - Transpose convolution from 256 to 128, batch normalization, ReLU, output size 8 by 8
  - Transpose convolution from 128 to 64, batch normalization, ReLU, output size 16 by 16
  - Transpose convolution from 64 to 1, Tanh, output size 32 by 32

## Discriminator

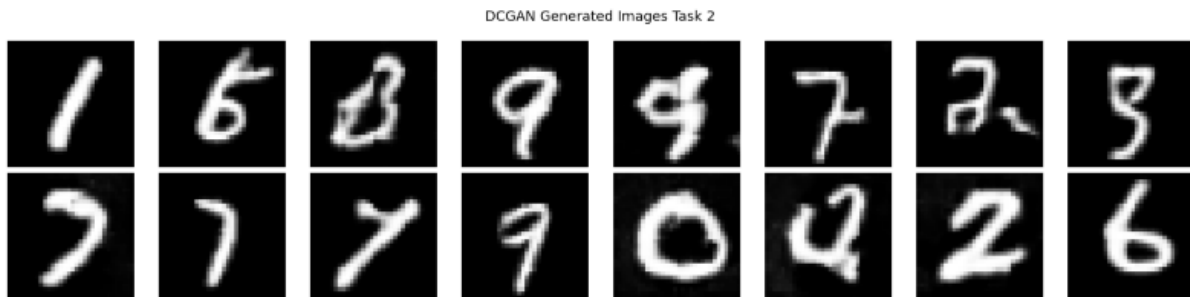
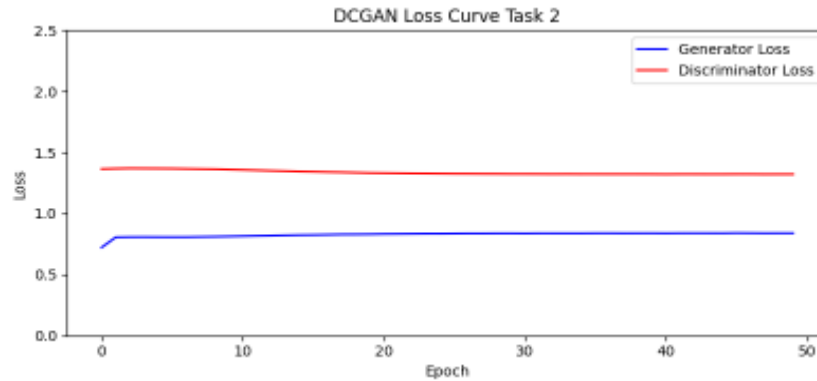
- The discriminator uses standard convolutions with spectral normalization to analyze the image and output a single real or fake probability.
  - Spectrally normalized convolution from 1 to 64, LeakyReLU, output 16 by 16
  - Spectrally normalized convolution from 64 to 128, LeakyReLU, output 8 by 8
  - Spectrally normalized convolution from 128 to 256, LeakyReLU, output 4 by 4
  - Spectrally normalized convolution from 256 to 1, Sigmoid, output 1 by 1

## Training Setup

- Binary Cross Entropy loss with label smoothing applied to real images.
- Adam optimizer for both networks with the generator learning at twice the rate of the discriminator.
- Trained for 50 epochs with a batch size of 128.

## Results

| Training: DCGAN Convolutional 50 Epochs |                |                |  |
|-----------------------------------------|----------------|----------------|--|
| Epoch [ 1/50]                           | D Loss: 1.3652 | G Loss: 0.7218 |  |
| Epoch [ 2/50]                           | D Loss: 1.3681 | G Loss: 0.8056 |  |
| Epoch [ 3/50]                           | D Loss: 1.3701 | G Loss: 0.8071 |  |
| Epoch [ 4/50]                           | D Loss: 1.3695 | G Loss: 0.8076 |  |
| Epoch [ 5/50]                           | D Loss: 1.3690 | G Loss: 0.8071 |  |
| Epoch [ 6/50]                           | D Loss: 1.3683 | G Loss: 0.8070 |  |
| Epoch [ 7/50]                           | D Loss: 1.3665 | G Loss: 0.8071 |  |
| Epoch [ 8/50]                           | D Loss: 1.3648 | G Loss: 0.8087 |  |
| Epoch [ 9/50]                           | D Loss: 1.3633 | G Loss: 0.8101 |  |
| Epoch [10/50]                           | D Loss: 1.3604 | G Loss: 0.8122 |  |
| Epoch [11/50]                           | D Loss: 1.3575 | G Loss: 0.8145 |  |
| Epoch [12/50]                           | D Loss: 1.3542 | G Loss: 0.8169 |  |
| Epoch [13/50]                           | D Loss: 1.3507 | G Loss: 0.8187 |  |
| Epoch [14/50]                           | D Loss: 1.3476 | G Loss: 0.8210 |  |
| Epoch [15/50]                           | D Loss: 1.3442 | G Loss: 0.8233 |  |
| Epoch [16/50]                           | D Loss: 1.3422 | G Loss: 0.8245 |  |
| Epoch [17/50]                           | D Loss: 1.3396 | G Loss: 0.8263 |  |
| Epoch [18/50]                           | D Loss: 1.3377 | G Loss: 0.8284 |  |
| Epoch [19/50]                           | D Loss: 1.3350 | G Loss: 0.8290 |  |
| Epoch [20/50]                           | D Loss: 1.3325 | G Loss: 0.8300 |  |
| Epoch [21/50]                           | D Loss: 1.3312 | G Loss: 0.8319 |  |
| Epoch [22/50]                           | D Loss: 1.3299 | G Loss: 0.8328 |  |
| Epoch [23/50]                           | D Loss: 1.3285 | G Loss: 0.8339 |  |
| Epoch [24/50]                           | D Loss: 1.3279 | G Loss: 0.8349 |  |
| Epoch [25/50]                           | D Loss: 1.3268 | G Loss: 0.8349 |  |
| Epoch [26/50]                           | D Loss: 1.3259 | G Loss: 0.8358 |  |
| Epoch [27/50]                           | D Loss: 1.3255 | G Loss: 0.8368 |  |
| Epoch [28/50]                           | D Loss: 1.3248 | G Loss: 0.8367 |  |
| Epoch [29/50]                           | D Loss: 1.3244 | G Loss: 0.8374 |  |
| Epoch [30/50]                           | D Loss: 1.3236 | G Loss: 0.8376 |  |
| Epoch [31/50]                           | D Loss: 1.3236 | G Loss: 0.8370 |  |
| Epoch [32/50]                           | D Loss: 1.3232 | G Loss: 0.8380 |  |
| Epoch [33/50]                           | D Loss: 1.3227 | G Loss: 0.8380 |  |
| Epoch [34/50]                           | D Loss: 1.3230 | G Loss: 0.8380 |  |
| Epoch [35/50]                           | D Loss: 1.3225 | G Loss: 0.8379 |  |
| Epoch [36/50]                           | D Loss: 1.3226 | G Loss: 0.8387 |  |
| Epoch [37/50]                           | D Loss: 1.3224 | G Loss: 0.8389 |  |
| Epoch [38/50]                           | D Loss: 1.3223 | G Loss: 0.8384 |  |
| Epoch [39/50]                           | D Loss: 1.3223 | G Loss: 0.8387 |  |
| Epoch [40/50]                           | D Loss: 1.3220 | G Loss: 0.8388 |  |
| Epoch [41/50]                           | D Loss: 1.3214 | G Loss: 0.8383 |  |
| Epoch [42/50]                           | D Loss: 1.3220 | G Loss: 0.8391 |  |
| Epoch [43/50]                           | D Loss: 1.3220 | G Loss: 0.8393 |  |
| Epoch [44/50]                           | D Loss: 1.3217 | G Loss: 0.8392 |  |
| Epoch [45/50]                           | D Loss: 1.3217 | G Loss: 0.8390 |  |
| Epoch [46/50]                           | D Loss: 1.3221 | G Loss: 0.8403 |  |
| Epoch [47/50]                           | D Loss: 1.3220 | G Loss: 0.8396 |  |
| Epoch [48/50]                           | D Loss: 1.3214 | G Loss: 0.8389 |  |
| Epoch [49/50]                           | D Loss: 1.3216 | G Loss: 0.8391 |  |
| Epoch [50/50]                           | D Loss: 1.3213 | G Loss: 0.8390 |  |



## Discussion

The DCGAN produced the best results of all three experiments. Both loss curves stayed remarkably flat throughout the 50 epochs, with the generator around 0.80 and the discriminator around 1.33. This kind of stability is exactly what spectral normalization is designed to achieve, and it clearly worked well here.

The generated images are noticeably crisper than anything produced by the fully connected models. The digits have clean outlines, proper proportions, and look like they were written with a consistent stroke. Numbers like 1, 2, 3, 4, 6, 7, and 9 are all clearly recognizable across the 16 samples, and there is good variety with no sign of the model getting stuck repeating the same image over and over. The convolutional layers allowed the generator to learn how to construct digits stroke by stroke rather than pixel by pixel, which is why the results look so much more natural and well formed.

## Overall Comparison

- Stable GAN training was achieved even with a simple fully connected network.
- Balanced learning rates and additional generator updates helped maintain stability.
- Adding deeper layers and batch normalization improved convergence speed.
- The deeper model generated cleaner and higher-quality images without instability.
- Convolutional layers in the DCGAN enabled better spatial feature learning.
- Spectral normalization allowed the DCGAN to train for 50 epochs without collapse.

## Conclusion

This tutorial covered the full journey from a basic GAN to a deep convolutional model, with each step introducing a concrete improvement. The base GAN established that the fundamental adversarial training idea works, the deeper model showed how architectural changes and normalization improve results, and the DCGAN demonstrated what becomes possible when you combine convolutional layers with proper initialization and spectral normalization. The most important takeaway is that GAN training is sensitive to balance. When the discriminator is too strong the generator cannot learn, and when it is too weak the generator produces lazy outputs. Getting that balance right through learning rate choices, update frequencies, and architectural constraints is what separates a working GAN from one that collapses.